Hyperbolic GCN for properties of Drug-like molecules Prediction

Ksenia Krasikova[1*†], **Payal Priyadarshini**[1*†], **Philipp Harlos**[1*†]

[1]Department of Mathematics and Informatics, FU Berlin, Berlin, DE

***For correspondence:**
ksek00@zedat.fu-berlin.de (KK);
payap02@zedat.fu-berlin.de (PP);
harlop97@zedat.fu-berlin.de (QQ)

[†]These authors contributed equally to this work

Abstract

Goal of the project: To implement a hyperbolic Graph Convolutional Network (GCN) to the problem of graph-level prediction of molecular properties of drug-like molecules using the QM9 dataset, and predicting the electric dipole moment.
Main results: A trained GCN model for predicting the molecular properties which showed an improved performance across all losses compared to last week's models.
Personal key learnings: To implement and train a hyperbolic graph neural network, specifically focusing on hyperbolic embeddings and graph convolutions for the prediction of molecular properties.
Estimated working hours: 11
Project evaluation: 4
Number of Words: 1502

Goal

This week's project focuses on the implementation of Hyperbolic Graph Convolutional Network (GCN). Using a provided code template and the QM9 benchmarking dataset, the objective is to train a GCN model to predict the dipole moment of molecules. In this context, each molecule is represented as a graph, where atoms serve as nodes and chemical bonds as edges.

Data

For this project, we utilised the QM9 dataset, which does not require manual download as it is directly accessible via **torch_geometric.datasets.QM9**. This simplifies the initial data acquisition, since the data is already provided as molecular graphs in a suitable structure. The dataset consists of approximately 133,000 molecules, each represented as a graph with nodes and edges. Nodes correspond to atoms, described by 11 features or dimentioanl vectors including one-hot encoded atom type(H,C,N,O,F), atomic number and other atomic properties. Edges represent chemical bonds and are described by a one-hot encoded vector for bond type. In addition to node and edge features, each graph also contains the adjacency matrix, 3D coordinates for each atom, and 19 target properties describing various chemical properties of the molecule. For this project, we specifically focus on one of these 19 regression targets, and the electric dipole moment at target index.

Pre-Processing

Following the initial data loading step, the QM9 data set undergoes specific preprocessing steps to further prepare it for the hyperbolic graph neural network. The main aim of this project was the prediction of electric dipole moment which is done using a **SetTarget** transform in order to isolate this property (target index 0) and discard any other regression targets. Once this is done, data is

then divided into training, validation and test subsets, limited to 1000 molecules.
The next step is converting the data from **torch_geometric.data** type into the **jraph.GraphsTuple** and set up graph batching, both done similarly to the last weeks project.
For a hyperbolic GCN, it is additionally required to embed the graph into hyperbolic space. To achieve this, we create a custom function that maps the node features of a given graph into a hyperbolic space of specified dimension. We utilize the **Morphomatics** library: first, we extract the node features, selecting the desired number of dimensions. Next, we set up the hyperbolic space and define its origin (the "pole," here at (0,0,...,1)). We then construct tangent vectors at the origin from the node features. Finally, we use the exponential map to project these tangent vectors, resulting in node embeddings that are valid points in hyperbolic space. Additionally, we visualize the hyperbolic embedding using Principal Component Analysis (PCA) to check correctness.

Methods

As a next step, we focus on a detailed explanation of the basic architecture of the hyperbolic graph neural network and the principles of equivariance and invariance.

Layers and Model

The core component of our model is the hyperbolic graph convolutional layer, implemented in the `GraphConvolution` class. This layer operates on a graph whose nodes have already been embedded in hyperbolic space, and utilizes Ψ (psi) and Φ (phi) as **nn.Dense** layers, following the design of previous project.
Due to the use of hyperbolic embeddings, the message processing and aggregation steps require adaptation. We first define input and output hyperbolic spaces, which may differ in dimension depending on the input feature size and the number of neurons in the dense layers. The embedded node features are then mapped to the Euclidean tangent space at the origin using the logarithmic map. As in the previous task, we use the psi layer to construct messages from sender nodes, applying a ReLU non-linearity as recommended in the lecture.
For message aggregation, we employ the mean rather than the sum, as sum aggregation resulted in numerical instability during training. The updated node representations h are then computed using the phi layer with an additional ReLU activation. Finally, these representations are mapped back to the hyperbolic output space using the exponential map.
The Graph Convolutional network model that has been executed in the GCN class is formulated with the help of stacking multiple layers of GC, same as in the last project. However, additionally, the initial node features are embedded into a hyperbolic space, before these are passed into a series of convolutional layers. Then a pooling is performed to combine the node features into a single value representation. This is done using the mean pooling strategy (**jraph.segment_mean**) because it is a common and effective strategy to create invariant graph representations. In the end, we get a modified GCN, which is used to predict the electric dipole moment from the hyperbolic graph embedding.

Equivariance and Invariance

A crucial consideration when designing graph neural networks is how they handle permutations of input nodes. The order in which nodes are presented should not be affecting fundamental structural properties or predictions, this leads to consideration of equivariance and invariance. Both Equivariance and Invariance serve as the important properties to test the model and to verify the geometric correctness of our implementation.
For both the equivariant-layer and invariant-model tests, we employed the same unit test framework as in last week's project. The equivariance test verifies whether the outputs of the hyperbolic convolution layer permute in the same way as the nodes are permuted in the input graph. Similarly, the invariance test for the hyperbolic GCN model checks whether the predicted dipole moment for a permuted graph remains identical to that of the original graph.

Training and Evaluation

We trained a hyperbolic graph convolutional network (GCN) within the Lorentz model of hyperbolic space, following the method proposed by Liu et al. ***Liu et al.*** (***2019***). The first layer maps 11-dimensional Euclidean atom features into 128-dimensional embeddings by applying the exponential map from the tangent space at the hyperbolic origin. Subsequent layers maintain this dimensionality while performing message passing directly in hyperbolic space. Model parameters were initialized using a random key (JAX PRNG key) and a sample batch from the training set. Optimization was carried out using the Adam algorithm with a learning rate of 1×10^{-4}, and training proceeded for 20 epochs, updating parameters based on the gradient of a chosen loss function. Identical to last week, we compared performance by training three different models using three different loss functions. As a baseline, we used the mean squared error (MSE), a standard choice for regression tasks such as predicting molecular dipole moments. We also tested mean absolute error (MAE), which penalizes all errors equally and is less sensitive to outliers, and the Huber loss, which blends the behavior of MSE and MAE for improved robustness. After each epoch, we evaluated model performance on the validation set to monitor generalization. While we tracked validation loss, we did not use it to adjust training (e.g., early stopping). Final performance was measured on a separate test set using all three loss functions. As expected, training loss generally decreased over time, and validation loss helped assess how well the model generalized to unseen data.

Results and Discussion

First, we examine the results of the hyperbolic embedding produced by the **embedd** function and the correctness of layer and model implementation using the unit tests. As shown in ***Figure A1***, several nodes overlap when projected into hyperbolic space. To investigate this phenomenon, we analyzed the node feature vectors used for embedding. We found that many of them are highly similar or identical, since most features are one-hot encoded. As a result, these nodes are mapped to the same points in hyperbolic space, resulting in their overlap in the PCA visualization. As for layer and model implementation, both passed the equivariance and invariance tests, so we deem all three implementations (embedding, convolutional layer, and resulting model) as usable.

After training, we evaluate the performance and learning dynamics of the model using three different loss functions, as shown in ***Figure A2***, ***Figure A3***, and ***Figure A4***. The training loss curves exhibit a rapid decline during the first epoch, followed by a relatively flat progression over the remaining 19 epochs, indicating early convergence. Although the training loss continues to decrease slowly in the later epochs, there is still a slight improvement; for example, the MSE loss decreases from approximately 2 to 1.5 by the end of training.

In contrast, the validation loss curves display greater fluctuations and do not exhibit a clear convergence trend, although a slight overall downward trend is observable. Additionally, we evaluate the model by predicting the dipole moment of unseen graphs from the test dataset, obtaining an MSE of approximately 1.27 D^2, an MAE of 0.88 D, and a Huber loss of 0.62 D. We compare these results to those obtained with last week's non-hyperbolic model. Compared to the previous model trained with MSE loss, the hyperbolic-space GCN surpasses its performance by approximately 3 D^2. For MAE, the hyperbolic model shows an imporvement of about 1.5 D. The Huber also improves with a small gain of about 0.6 D in the hyperbolic setting.

Taken together, these results, both from the training and validation loss curves and test set evaluation, suggest that the hyperbolic model may be better suited for the dipole moment prediction task. This may be attributed to the geometry of hyperbolic space, which can more effectively capture the relationships present in molecular structures.

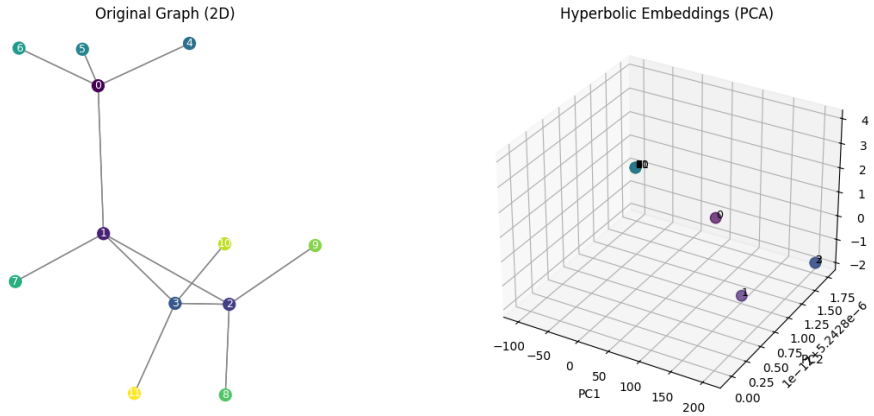
Contributions

1. Worked on baseline implementation, writing Results and Discussion section and editing other sections by Ksenia Krasikova.
2. Pre-processing section , Methods section (model, layers and equivariance) and minor changes by Payal Priyadarshini.
3. Worked on baseline implementation, writing Method Section by Philipp Harlos.

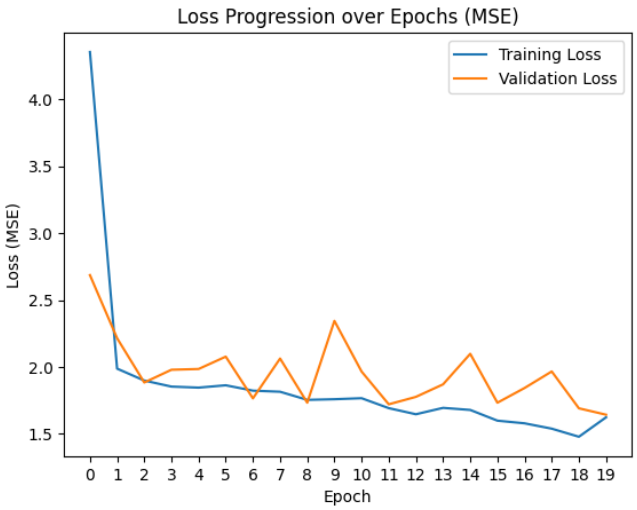
References

Liu Q, Nickel M, Kiela D. Hyperbolic Graph Neural Networks. In: Wallach H, Larochelle H, Beygelzimer A, d'Alché-Buc F, Fox E, Garnett R, editors. *Advances in Neural Information Processing Systems*, vol. 32 Curran Associates, Inc.; 2019. https://proceedings.neurips.cc/paper_files/paper/2019/file/103303dd56a731e377d01f6a37badae3-Paper.pdf.

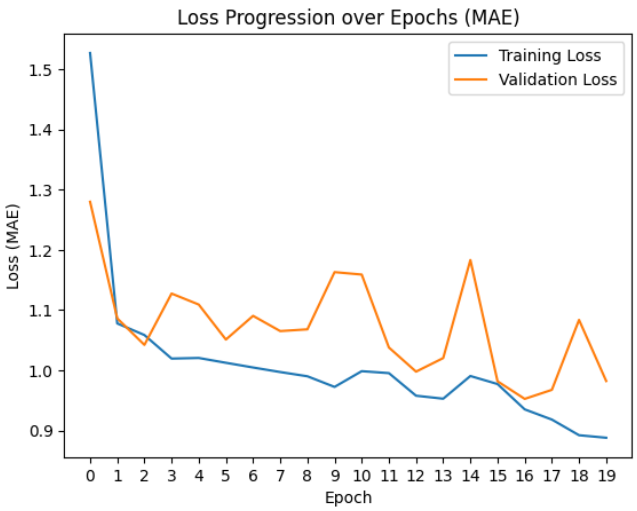
Supplementary Figures



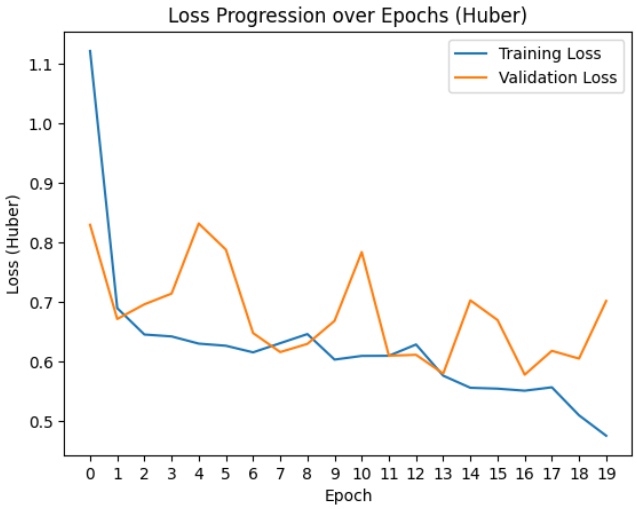
Appendix 0—figure A1. Left: Original molecular graph structure in 2D, with nodes colored and labeled by index. **Right:** Corresponding node embeddings visualized in 3D using Principal Component Analysis (PCA) after embedding the graph into hyperbolic space.



Appendix 0—figure A2. Training and validation loss curves (MSE) for the hyperbolic GCN model over 20 epochs.



Appendix 0—figure A3. Training and validation loss curves (MAE) for the hyperbolic GCN model over 20 epochs.



Appendix 0—figure A4. Training and validation loss curves (Huber) for the hyperbolic GCN model over 20 epochs.